

Computational Complexity

Ref: CLRS 3rd edition (Chapter 34 initial portion) and
Algorithm Design (Kleinberg+Tardos)

Adapted from the slide prepared by Dr. Md. Shamsuzzoha Bayzid

Introduction

- Polynomial-time algorithms $\Rightarrow$ on inputs of size n , the worst-case running time is $O(n^k)$ for some **constant** k .
- There are problems, such as Turing's famous "Halting Problem," that cannot be solved by any computer, no matter how much time we allow.
- There are also problems that can be solved, but not in time $O(n^k)$ for any constant k .
- Generally, we think of problems that are solvable by polynomial-time algorithms as being **tractable, or easy**, and problems that require super-polynomial time as being **intractable, or hard**.

“NP-complete” problems

- No polynomial-time algorithm has yet been discovered for an NP-complete problem, nor has anyone yet been able to prove that no polynomial-time algorithm can exist for any one of them.
- This so-called $P \neq NP$ question has been one of the deepest, most perplexing open research problems in theoretical computer science since it was first posed in 1971.

Comparison Between Similar Problems

- In each of the following pairs of problems, one is solvable in polynomial time and the other is NP-complete, but the difference between problems appears to be slight:
- Shortest vs. longest simple paths:
 - Even with negative edge weights, we can find shortest paths from a single source in a directed graph $G = (V, E)$ in $O(VE)$ time.
 - Finding a longest simple path between two vertices is difficult, however.
 - Merely determining whether a graph contains a simple path with at least a given number of edges is NP-complete.

Comparison Between Similar Problems (contd.)

- Euler tour vs. hamiltonian cycle:
 - An Euler tour of a connected, directed graph $G = (V, E)$ is a cycle that traverses each edge of G exactly once, although it is allowed to visit each vertex more than once.
 - We can determine whether a graph has an Euler tour in only $O(E)$ time and, in fact, we can find the edges of the Euler tour in $O(E)$ time.
 - A hamiltonian cycle of a directed graph $G = (V, E)$ is a simple cycle that contains each vertex in V .
 - Determining whether a directed graph has a hamiltonian cycle is NP-complete.
 - Determining whether an undirected graph has a hamiltonian cycle is also NP-complete.

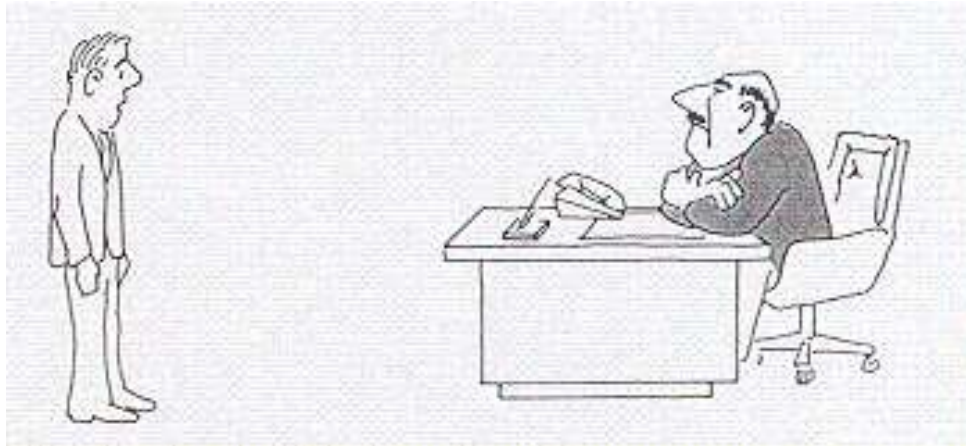
Comparison Between Similar Problems (contd.)

- 2-CNF satisfiability vs. 3-CNF satisfiability:
 - A boolean formula contains variables whose values are 0 or 1; boolean connectives such as $\wedge$ (AND), $\vee$ (OR), and $\neg$ (NOT); and parentheses.
 - A boolean formula is satisfiable if there exists some assignment of the values 0 and 1 to its variables that causes it to evaluate to 1.
 - A boolean formula is in k-conjunctive normal form, or k-CNF, if it is the AND of clauses of ORs of exactly k variables or their negations. For example, the boolean formula $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$ is in 2-CNF. (It has the satisfying assignment $x_1 = 1; x_2 = 0; x_3 = 1$.)
 - Although we can determine in polynomial time whether a 2-CNF formula is satisfiable, determining whether a 3-CNF formula is satisfiable is NP-complete.

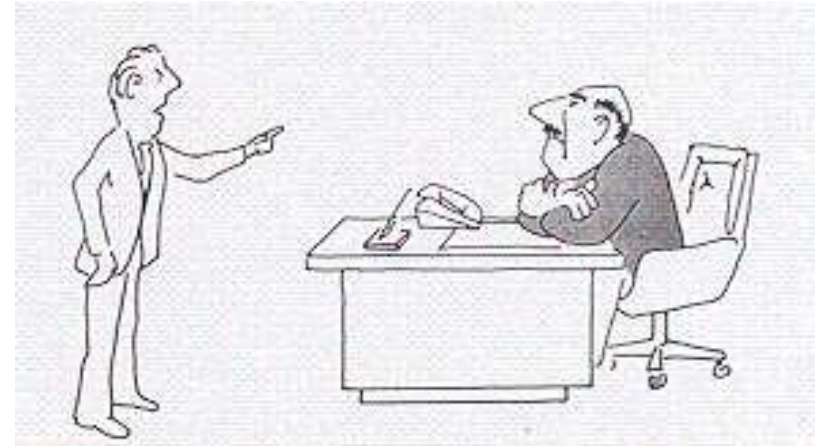
Computational Complexity

- It is really hard to prove that there is **no efficient algorithm** for a particular problem.
- What to do?
 - Show that the problem is as hard as many other problems that have been worked on by a host of brilliant scientists over a very long time, yet there is no efficient solutions.
- A fundamental aspect of complexity theory is categorizing problems into various equivalence classes.
 - Comparing **relative difficulty** of different problems

Computational Complexity (contd.)



I cannot solve it efficiently **because** I am **dumb**!



I cannot find an efficient
solution **because** there **isn't any**!

Class P

- The class P consists of those problems that are solvable in polynomial time.
- More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k , where n is the size of the input to the problem.

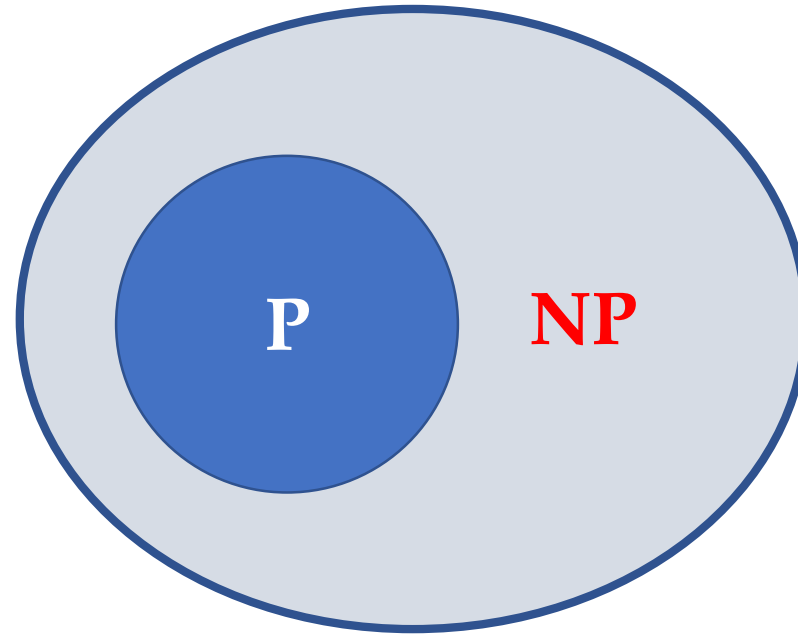
Class NP

- The class NP consists of those problems that are “verifiable” in polynomial time.
 - If we were somehow given a “certificate” of a solution, then we could verify that the certificate is correct in time polynomial in the size of the input to the problem.
- For example, in the Hamiltonian cycle problem, given a directed graph $G = (V, E)$, a certificate would be a sequence of $|V|$ vertices.
- We could easily check in polynomial time whether each edge of the sequence is in E and whether we can return to the source vertex again.
- As another example, for 3-CNF satisfiability, a certificate would be an assignment of values to variables.
- We could check in polynomial time that this assignment satisfies the boolean formula.

P vs NP

- Any problem in P is also in NP, since if a problem is in P then we can solve it in polynomial time without even being supplied a certificate.
- $P \subseteq NP$.
- The open question is whether or not P is a proper subset of NP.
 - In other words, whether $P=NP$ or not

What we think the world looks like



Complement of a problem

For every problem X , there is a natural *complementary* problem X'

If the answer to X is “YES”, the answer to X' is “NO”

- SAT: Is a given boolean formula in CNF **satisfiable**?
- UNSAT: Is a given boolean formula in CNF **unsatisfiable**?

Complement of a problem

$$x \in P \Rightarrow x' \in P$$

We have an efficient algorithm A to solve X.
We can design an algorithm **A' for X'** which **runs A**
and **flip the answer**

Complement of a problem

Is it true that $x \in \text{NP} \Rightarrow x' \in \text{NP}$?

Just because you have an efficient certifier for X **does not mean** that you can design an **efficient certifier for X'**

YES answers verification involves “**there exist**” quantifier, whereas **NO** answer verification involves “**for all**”

co-NP

$$x \in \text{NP} \Rightarrow x' \in \text{co-NP}$$

- co-NP: Set of decision problems with a polynomial time verification for a “NO” answer
 - Just the opposite to NP
- UNSAT: Is a given Boolean formula unsatisfiable?
 - UNSAT $\in$ co-NP

co-NP

- **PRIME**: Is a given number x prime?
 - NO answer verification is easy; PRIME is in co-NP
 - YES answer verification does not seem to be easy
- **PRIME** is in P (Agrawal, Kayal, and Saxena (AKS))

NP and co-NP

Does $NP = co-NP$?

We don't know yet!

What is the relationship between P and $co-NP$?

P is a subset of $co-NP$

$x \in P \Rightarrow x \in co-NP$

If the answer is NO for a problem and you can solve it in polynomial time, you can verify the NO answer without looking at the certificate

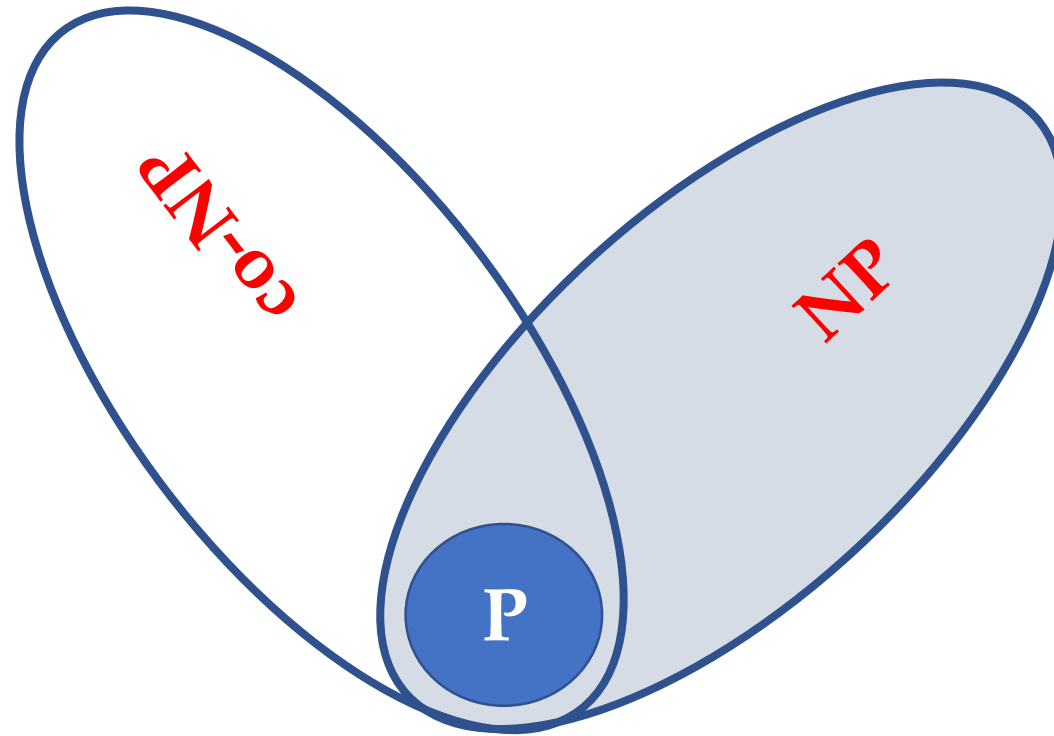
NP and co-NP

What is the relationship between P and co-NP?

P is a subset of co-NP

$$x \in P \Rightarrow x' \in P \Rightarrow x' \in NP \Rightarrow x \in \text{co-NP}$$
$$x \in P \Rightarrow x \in \text{co-NP}$$

What we think the world looks like



P, NP, co-NP

Just like P and NP, the widespread belief is that

$NP \neq co-NP$

But we don't know!

Proving $NP \neq co-NP$ is even a bigger step than proving $P \neq NP$

P, NP, co-NP

$$\text{NP} \neq \text{co-NP} \Rightarrow \text{P} \neq \text{NP}$$

$$\text{P} = \text{NP} \Rightarrow \text{NP} = \text{co-NP} \text{ (contrapositive of the above)}$$

If $\text{P} = \text{NP}$, then

NP is a subset of co-NP

$$x \in \text{NP} \Rightarrow x \in \text{P} \Rightarrow x' \in \text{P} \Rightarrow x' \in \text{NP} \Rightarrow x \in \text{co-NP}$$

co-NP is a subset of NP

$$x \in \text{co-NP} \Rightarrow x' \in \text{NP} \Rightarrow x' \in \text{P} \Rightarrow x \in \text{P} \Rightarrow x \in \text{NP}$$

So $\text{NP} = \text{co-NP}$

NP and co-NP Example

- Travelling salesman problem: Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?
- Decision version of TSP: "Given a graph G , is there a path of length L or less in G that visits each vertex at least once?" is in NP.
 - If I say "yes, there is a path of length L or less that visits each vertex at least once"
 - The way I prove that is by giving you a path of length L or less that visits each vertex at least once, and the way you check my solution is by taking my path, checking that it travels to each vertex at least once, and that it's of length L or less.

NP and co-NP Example (contd.)

- The complement of this problem would be "Given a graph G , are there no paths of length L or less in G that visit each vertex at least once?"
 - Verifying "yes" answer ("yes, there are no paths") to this question is hard, because we have to exhaustively check all the paths.
 - Verifying "no" answer ("no, there are some paths") to this question is easy, because we need only one path to verify it, just like NP.
 - To prove it, you are given a path of length L or less that visits each vertex at least once. So it is not true that "there are no paths in G of length L that visit each vertex at least once."
- So here the original problem is in NP and the complement problem is in co-NP, as it always should be.

NP and co-NP Example (contd.)

- The SAT problem asks "does there exist a boolean assignment which makes this boolean formula evaluate to True?".
- The complement problem, which is in co-NP, asks, "do all boolean assignments make this formula evaluate to False?"

Integer Factorization

- An example of a problem that is known to belong to both NP and co-NP (but not known to be in P) is integer factorization.
- Integer factorization – optimization version: Given a positive integer m , find all integer factors of m .
- Integer factorization – decision version: Given positive integers m and n , determine if m has a factor less than n and greater than 1.
- Membership in NP is clear.
 - You are given a factor, check whether it is a factor of m and whether it is less than n .
- Membership in co-NP is also straightforward.
 - You are given some factors of m , all greater or equal to n .
 - You have to verify that all of them are indeed prime (by using **AKS primality test**) and their multiplication results in m .

Integer Factorization (contd.)

- It is presently not known whether there is a polynomial-time algorithm for integer factorization (equivalently that integer factorization is in P), and hence this example is interesting as one of the most natural problems known to be in NP and co-NP but not known to be in P.

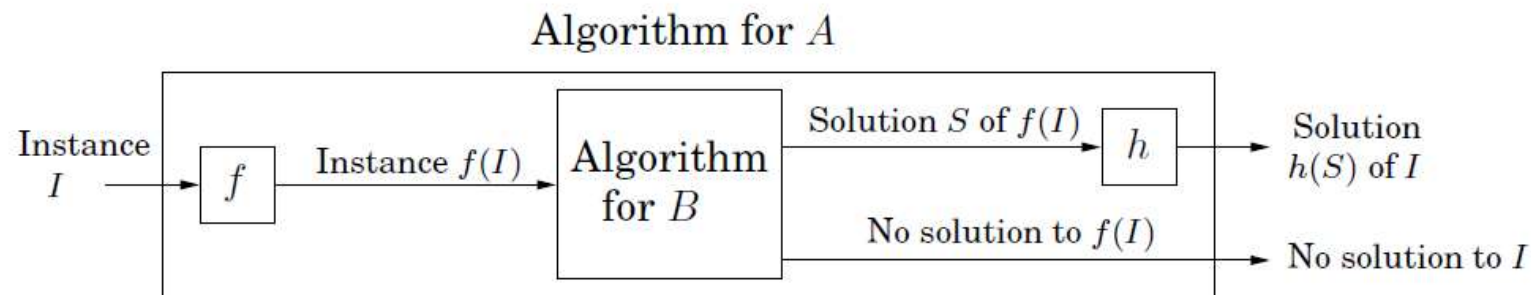
Polynomial-time Reductions

Reduction

$$A \leq_p B$$

Problem A is polynomial time reducible to problem B

If $A \leq_p B$ and if we have a blackbox that solves B, then we can solve A by reducing an instance of A into an instance of B

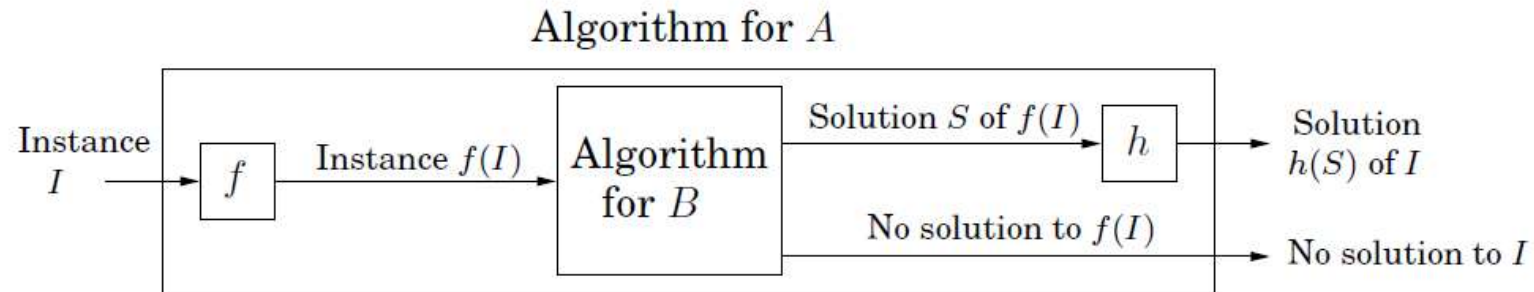


Polynomial-time Reductions

Reduction

$$A \leq_p B$$

Problem A is polynomial time reducible to problem B



A is at least as hard as B



B is at least as hard as A

Polynomial-time Reductions

Reduction

$$A \leq_p B$$

Problem A is polynomial time reducible to problem B

If you **can solve** B in polynomial time,
then you **can solve** A in polynomial time

If you **cannot solve** A in polynomial time,
then you **cannot solve** B in polynomial time

Difficulty flows from A to B;
Efficiency flows from B to A

Class NP-hard

- A problem is in the class NP-hard if it is as “hard” as any problem in NP.
 - The problem itself may or may not be in class NP.

NP-hard problems

NP-hard

Problem X is an NP-hard problem if
for **all** $Y \in \text{NP}$, $Y \leq_p X$

If a problem X is ***NP-hard***, a polynomial-time algorithm for X would
imply a polynomial-time algorithm
for *every problem in NP*.

Class NP-complete

- A problem is in the class NP-complete (NPC)—if it is in NP and is as “hard” as **any problem** in NP.
- If any NP-complete problem can be solved in polynomial time, then every problem in NP has a polynomial time algorithm.

NP-complete problems

NP-complete

Problem X is an NP-complete problem if:

- i) $X \in \text{NP}$
- ii) for **all** $Y \in \text{NP}$, $Y \leq_p X$

NP-complete problems are the **hardest** problems in NP

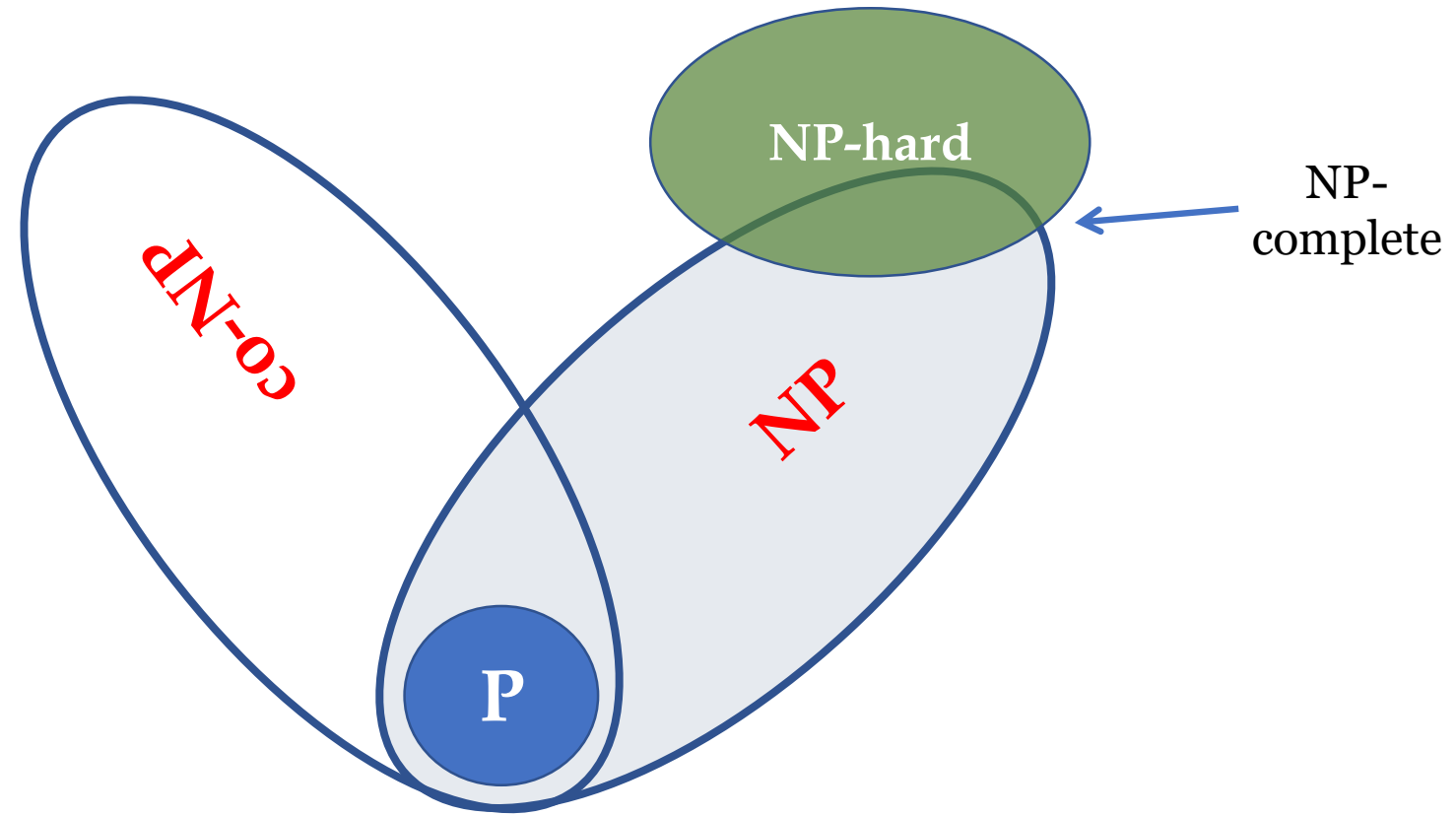
NP-complete is a set of problems with the “same level of” difficulty

Same level of difficulty because each problem in NPC can be reduced to others. So they define both upper bound and lower bound on the difficulty. However, NP-hard problems define a lower bound. Because it is possible that one NP-hard cannot be reduced to another NP-hard.

NP-complete Problems

- If you can establish a problem as NP-complete or NP-hard, you provide good evidence for its intractability.
 - As an engineer, you would then do better to spend your time developing an approximation algorithm or solving a tractable special case, rather than searching for a fast algorithm that solves the problem exactly.
 - When we demonstrate that a problem is NP-complete or NP-hard, we are making a statement about how hard it is (or at least how hard we think it is), rather than about how easy it is.

What we think the world looks like



Three Key Concepts

We rely on three key concepts in showing a problem to be NP-complete:

- Decision problems vs. optimization problems
- Reductions
- A first NP-complete problem

Decision Problems vs. Optimization Problems

- Optimization problems: each feasible (i.e., “legal”) solution has an associated value, and we wish to find a feasible solution with the best value.
 - Shortest path: we wish to find a path from u to v that uses the fewest edges.
- NP-completeness applies directly **not to optimization problems**, however, but to **decision problems**, in which the answer is simply “yes” or “no” (or, more formally, “1” or “0”).

Decision Problems vs. Optimization Problems (contd.)

- We usually can cast a given optimization problem as a related decision problem by imposing a bound on the value to be optimized.
 - For example, a decision problem related to SHORTEST-PATH is PATH: given a directed graph G , vertices u and v , and an integer k , does a path exist from u to v consisting of at most k edges?
- The decision problem is in a sense “easier,” or at least “no harder” than the related optimization problem.
 - As a specific example, we can solve PATH by solving SHORTEST-PATH and then comparing the number of edges in the shortest path found to the value of the decision-problem parameter k .

Decision Problems vs. Optimization Problems (contd.)

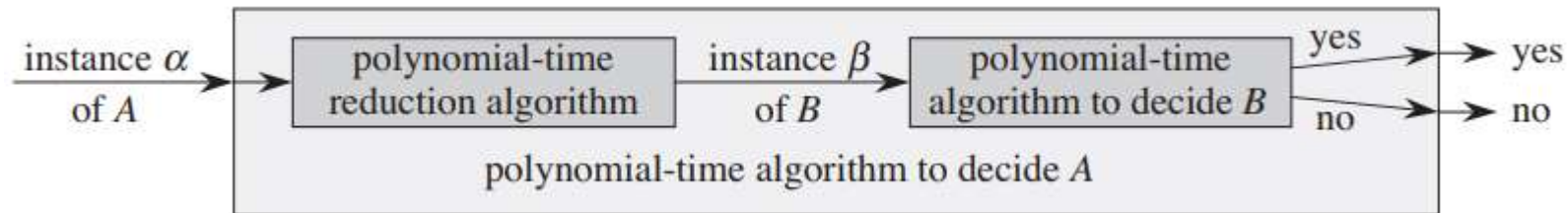
- If an optimization problem is easy, its related decision problem is easy as well.
- If we can provide evidence that a decision problem is hard, we also provide evidence that its related optimization problem is hard.
- Thus, even though it restricts attention to decision problems, the theory of NP-completeness often has implications for optimization problems as well.

Reductions

- Showing that one problem is no harder or no easier than another applies even when both problems are decision problems.
- Let us consider a decision problem A , which we would like to solve in polynomial time.
- We call the input to a particular problem an **instance** of that problem.
 - For example, in PATH, an instance would be a particular graph G , particular vertices u and v of G , and a particular integer k .
- Now suppose that we already know how to solve a different decision problem B in polynomial time.

Reductions (contd.)

- Finally, suppose that we have a procedure that transforms any instance α of A into some instance β of B with the following characteristics:
 - The transformation takes polynomial time.
 - The answers are the same. That is, the answer for α is “yes” if and only if the answer for β is also “yes.”
- We call such a procedure a **polynomial-time reduction algorithm**.

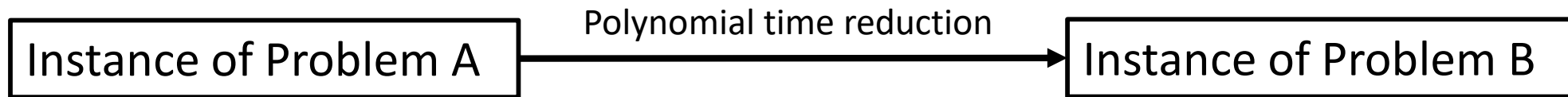


Reductions (contd.)

- By “reducing” solving problem A to solving problem B, we use the “easiness” of B to prove the “easiness” of A.
- NP-completeness is about showing how hard a problem is rather than how easy it is.
- How could we use polynomial-time reductions to show that no polynomial-time algorithm can exist for a particular problem B?
 - Suppose we have a decision problem A for which we already know that no polynomial-time algorithm can exist.
 - Suppose further that we have a polynomial-time reduction transforming instances of A to instances of B.
 - Now we can use a simple proof by contradiction to show that no polynomial time algorithm can exist for B.

Reductions (contd.)

- For NP-completeness, we **cannot** assume that there is absolutely no polynomial time algorithm for problem A.
- The proof methodology is similar, however, in that we prove that **problem B is NP-complete on the assumption that problem A is also NP-complete.**



If problem A is known to be NP-complete, problem B is also NP-complete here

Proof of NP-completeness

You want to prove that problem X is NP-complete.

NP-complete

You have to prove that X is in NP and
for all problems $Y \in \text{NP}$, $Y \leq_p X$

Alternative: Prove that X is in NP and
a known NP-complete problem $Y \leq_p X$

A First NP-complete Problem

- We can use the circuit-satisfiability problem.
 - We are given a boolean combinational circuit composed of AND, OR, and NOT gates, and we wish to know whether there exists some set of Boolean inputs to this circuit that causes its output to be 1.

Vertex Cover

A vertex cover of a graph is a set S of vertices/nodes such that every edge has at least one endpoint in S .

We try to “cover” each of the edges by choosing at least one of its vertices.

For a given graph $G=(V,E)$, V is always a vertex cover. It is difficult to find the **smallest vertex cover**.

Vertex Cover: Decision version

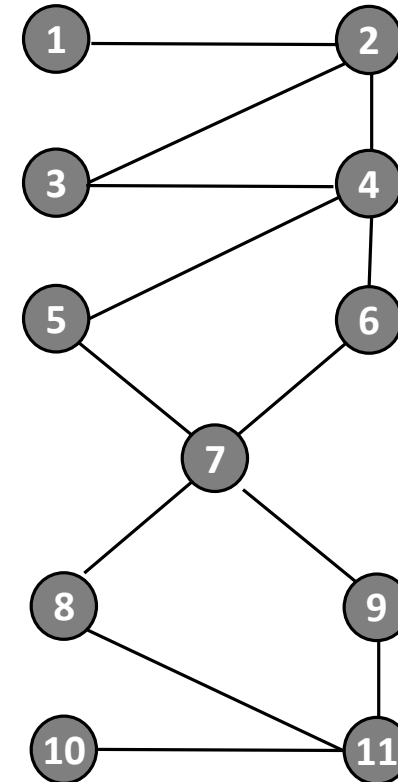
Given a graph G and a number k , does G contain a vertex cover of size **at most** k ?

Vertex Cover

Vertex Cover

Given a graph G and a number k , does G contain a vertex cover of size at most k ?

► Is there a vertex cover of size 4?



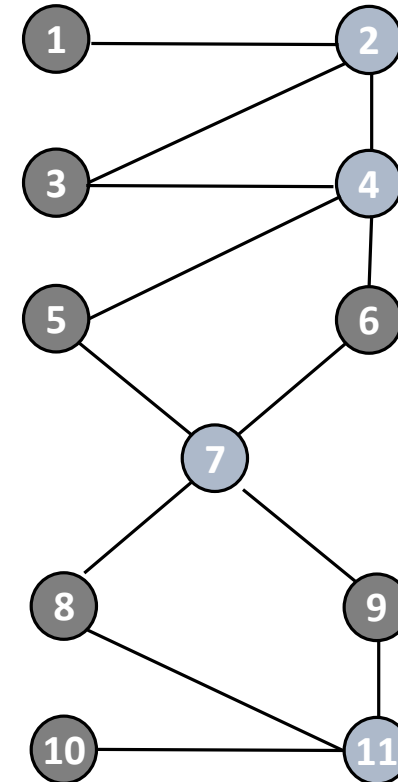
Vertex Cover

Vertex Cover

Given a graph G and a number k , does G contain a vertex cover of size at most k ?

► Is there a vertex cover of size 4?

► Yes

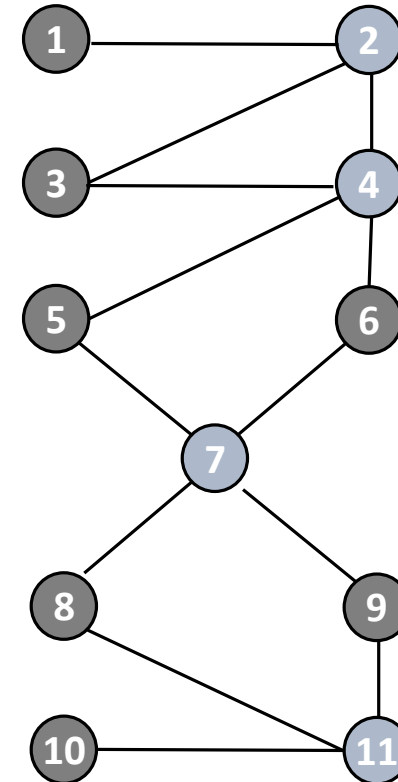


Vertex Cover

Vertex Cover

Given a graph G and a number k , does G contain a vertex cover of size at most k ?

- ▶ Is there a vertex cover of size 4?
 - ▶ Yes
- ▶ Is there a vertex cover of size 3?
 - ▶ No



Independent Set

An independent set of a graph is a set I of nodes such that they are pairwise non-adjacent.

In trivial case, I may contain only one vertex. It is difficult to find the **largest** I .

Independent Set: Decision version

Given graph G and a number k , does G contain a set of **at least** k independent vertices?

Independent Set: Optimization vs Decision

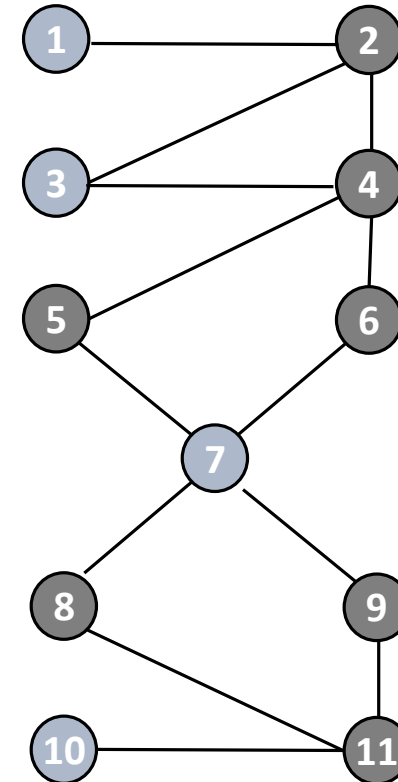
- If we can solve the decision version of Independent Set for every k , we can also find a maximum independent set.
 - Using binary search, we have to try for only $O(\log |V|)$ different values of k .

Independent Set

Independent Set

Given graph G and a number k , does G contain a set of at least k independent vertices?

- ▶ Is there an independent set of size 4?
 - ▶ Yes

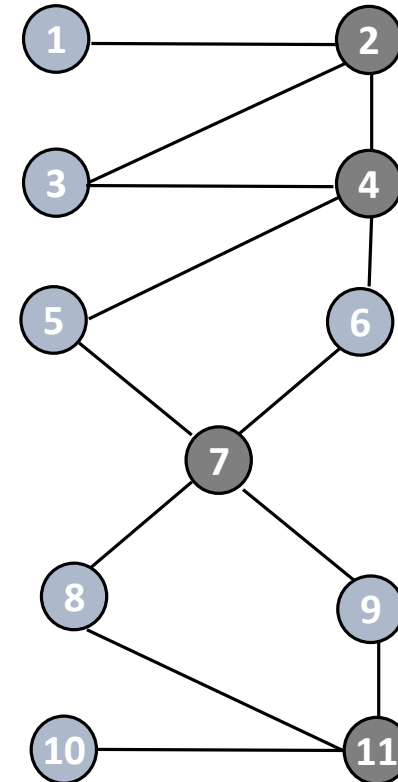


Independent Set

Independent Set

Given graph G and a number k , does G contain a set of at least k independent vertices?

- ▶ Is there an independent set of size 7?
 - ▶ Yes

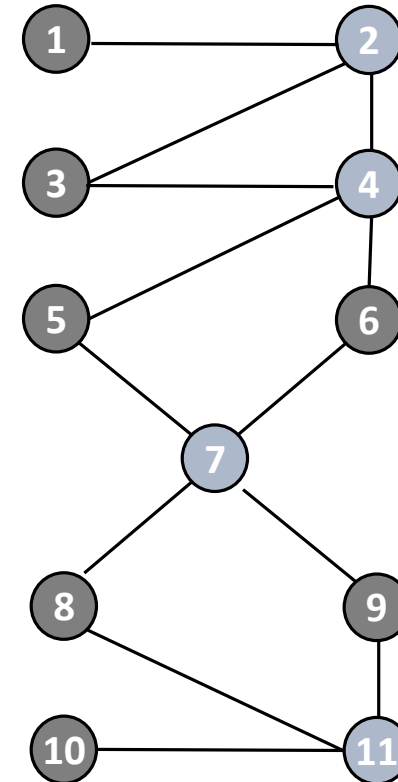


Vertex Cover

Vertex Cover

Given a graph G and a number k , does G contain a vertex cover of size at most k ?

- ▶ Is there a vertex cover of size 4?
 - ▶ Yes
- ▶ Is there a vertex cover of size 3?
 - ▶ No



Independent Set to Vertex Cover

Independent Set

Given graph G and a number k , does G contain a set of at least k independent vertices?



Vertex Cover

Given a graph G and a number k , does G contain a vertex cover of size at most k ?

Independent Set to Vertex Cover (and vice versa)

Theorem

If $G = (V, E)$ is a graph, then S is an independent set **if and only if** $V - S$ is a vertex cover.

Proof. First, suppose that S is an independent set. Consider an arbitrary edge $e = (u, v)$. Since S is independent, it cannot be the case that both u and v are in S ; so one of them must be in $V - S$. It follows that every edge has at least one end in $V - S$, and so $V - S$ is a vertex cover.

Conversely, suppose that $V - S$ is a vertex cover. Consider any two nodes u and v in S . If they were joined by edge e , then neither end of e would lie in $V - S$, contradicting our assumption that $V - S$ is a vertex cover. It follows that no two nodes in S are joined by an edge, and so S is an independent set.

Relationship between Independent Set and Vertex Cover

So

Independent set $\leq_p$ Vertex cover

Vertex cover $\leq_p$ Independent set

Both are in the same level of difficulty.

3-SAT to Independent Set

Input

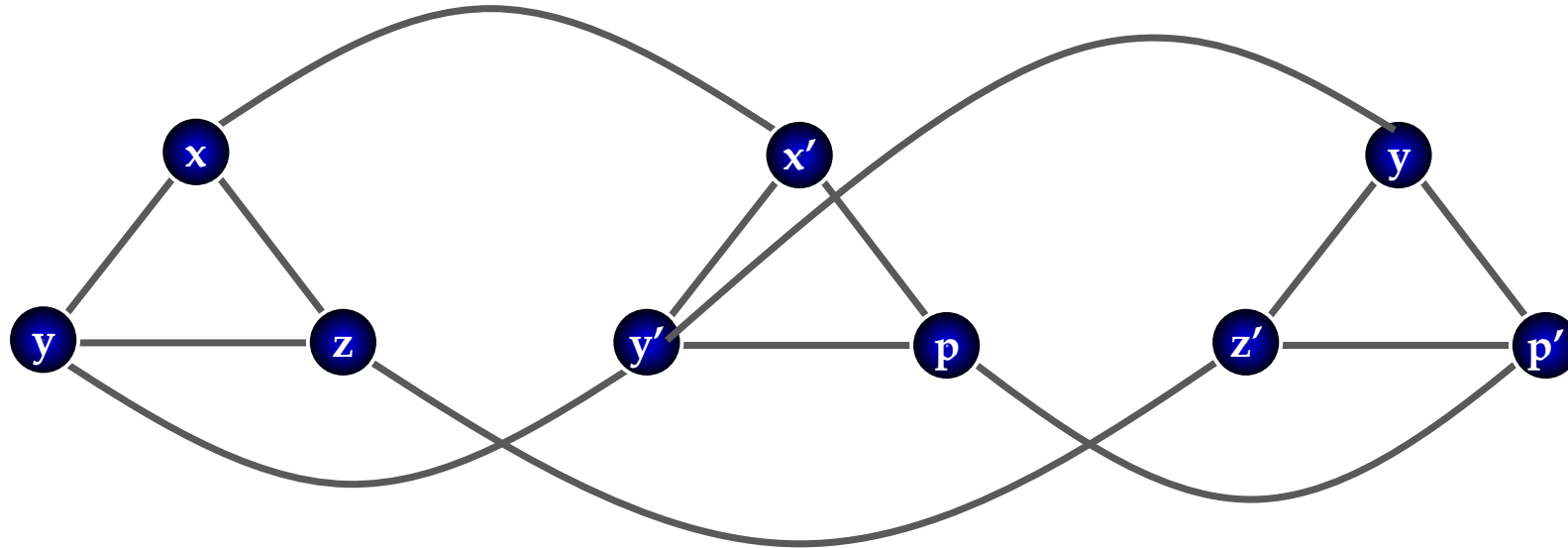
An instance of 3-SAT with $X = \{x_1, x_2, \dots, x_n\}$ variables
and k clauses $C_1, C_2, \dots, C_k$

$$(x \vee y \vee z) \wedge (x' \vee y' \vee p) \wedge (y \vee z' \vee p')$$

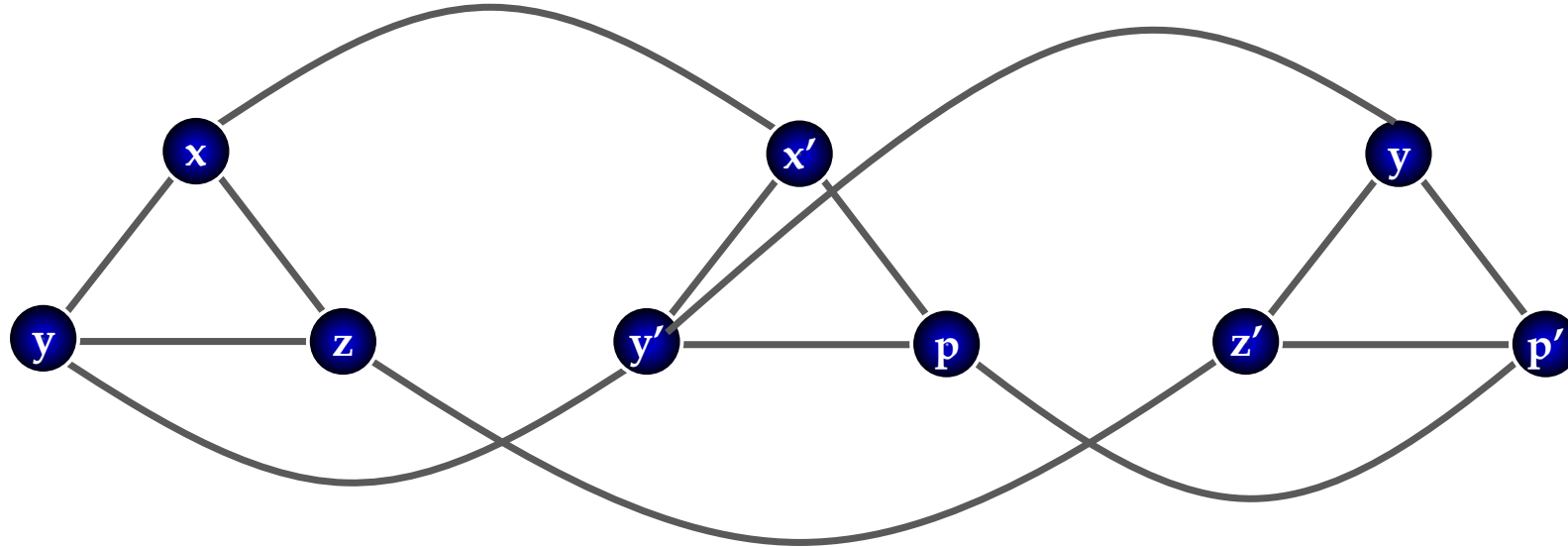
3-SAT to Independent Set

- **Non-trivial reduction** as the problems are quite different in nature
- A **clique** (triangle for a clause with three variables) for every clause
- An **edge** between x and x' in two different clauses

$$(x \vee y \vee z) \wedge (x' \vee y' \vee p) \wedge (y \vee z' \vee p')$$



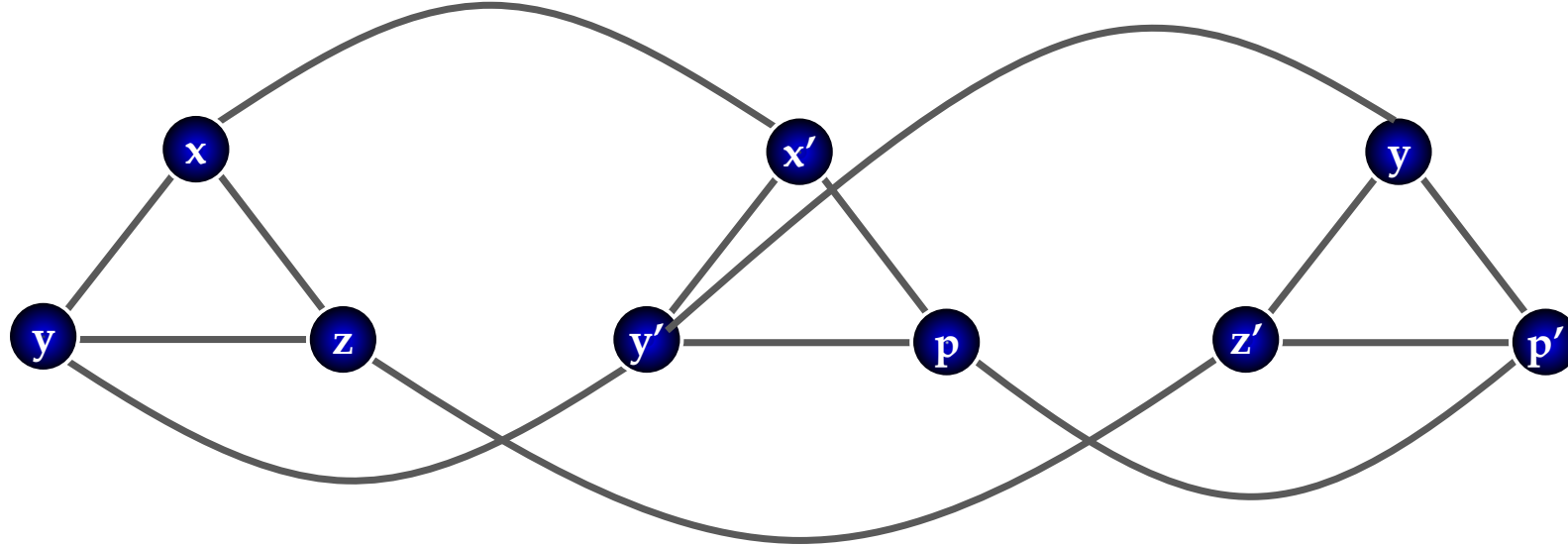
3-SAT to Independent Set



Satisfiable $\Rightarrow \exists$ an independent set

- If the instance of 3-SAT is **satisfiable**, then **each triangle** in our graph contains **at least one node** which evaluates to 1
- Let S be a set consisting of **one such node from each triangle**. Then S is an **independent set**. Because, if S contains two adjacent vertices then both of them cannot be 1. So **there is no pair of vertices having a common edge** in S .

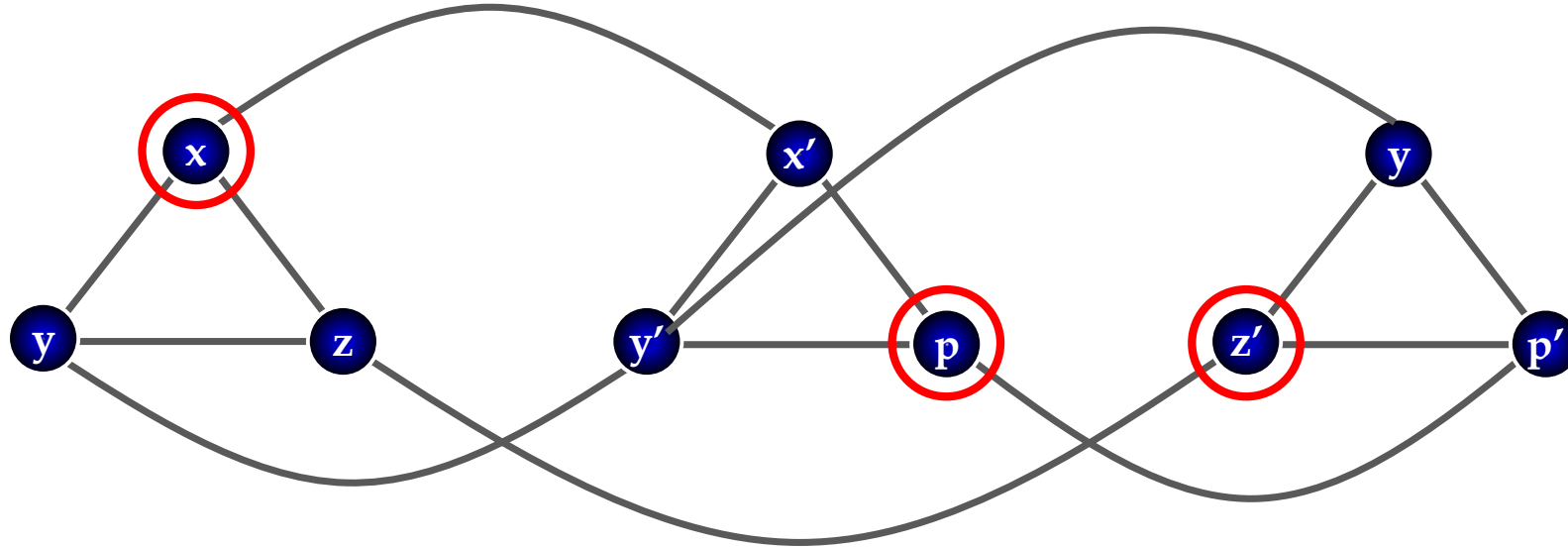
3-SAT to Independent Set



$\exists$ an independent set of size $k \Rightarrow$ Satisfiable

- First, we claim that $|S| = k$ (exactly k and it contains exactly one node from each clause). This is true because if you pick more than a two nodes from a triangle, it will not be an independent set.
- Next, we prove that we can assign values (0/1) to the variables in the instance of 3-SAT problem so that it evaluates to 1

3-SAT to Independent Set



$\exists$ an independent set of size $k \Rightarrow$ Satisfiable

- If x_i is present in the independent set (IS), $x_i = 1$
- If x'_i is present in the independent set (IS), $x_i = 0$
- If both x'_i and x_i are absent, assign any value to x_i (doesn't matter)
- In this example, $\{x, p, z'\}$ is an IS.
 $x = 1, p = 1, z = 0$ will satisfy the formula

Vertex Cover to Set Cover

Vertex Cover

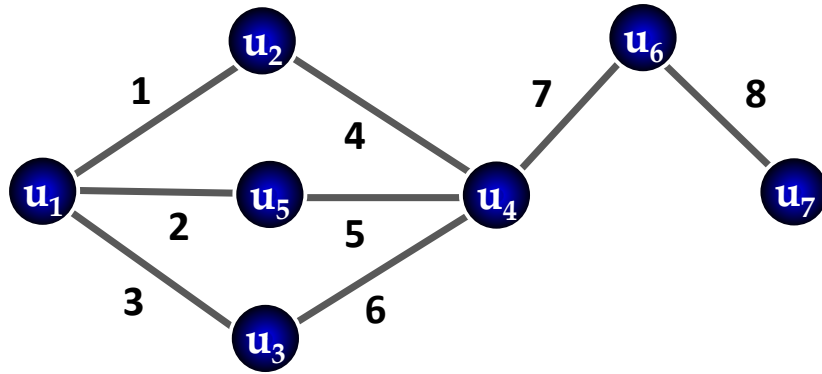
Given a graph G and a number k , does G contain a vertex cover of size **at most** k ?



Set Cover

Given a set U of n elements, a collection of $S_1, S_2, \dots, S_m$ of subsets of U , is there a collection of **at most** k of these subsets whose union equals U ?

Vertex Cover to Set Cover



$U = \{1, 2, 3, 4, 5, 6, 7, 8\}$

$S_1 = \{1, 2, 3\}$

$S_2 = \{1, 4\}$

$S_3 = \{3, 6\}$

$S_4 = \{4, 5, 6, 7\}$

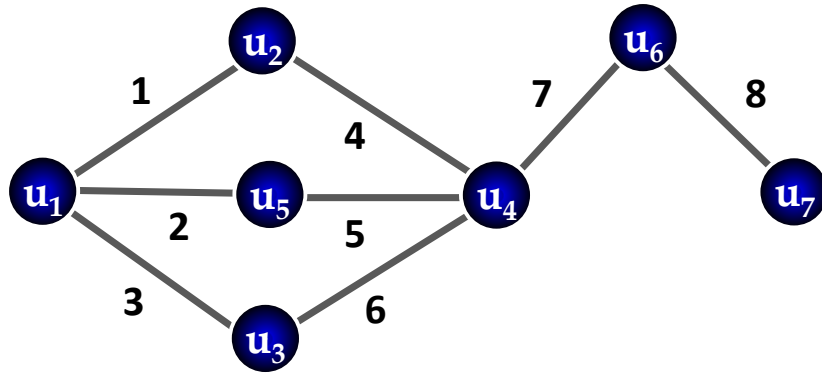
$S_5 = \{2, 5\}$

$S_6 = \{7, 8\}$

$S_7 = \{8\}$

- $U = E$ (the set of edges)
- For each vertex u_i , create a subset S_i such that S_i contains all the edges incident to u_i

Vertex Cover to Set Cover



$U = \{1, 2, 3, 4, 5, 6, 7, 8\}$

$S_1 = \{1, 2, 3\}$

$S_2 = \{1, 4\}$

$S_3 = \{3, 6\}$

$S_4 = \{4, 5, 6, 7\}$

$S_5 = \{2, 5\}$

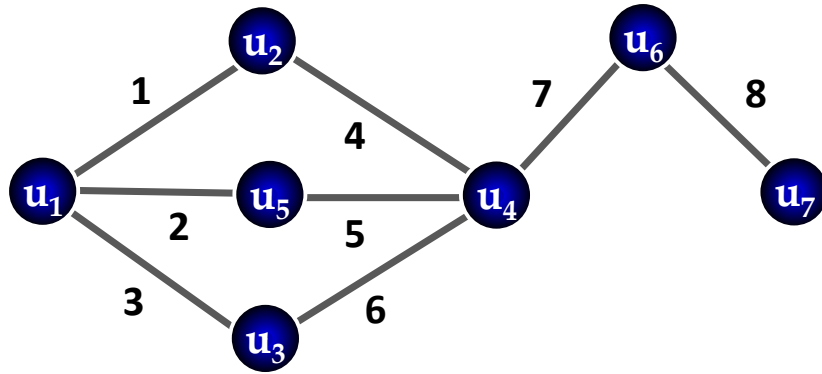
$S_6 = \{7, 8\}$

$S_7 = \{8\}$

$\exists$ a **set cover** of size at most $k \Rightarrow \exists$ a **vertex cover** of size at most k

- If $S_1, S_2, \dots, S_l$; $1 \leq l \leq k$ is a set cover of U , then every edge in G is incident to one of the vertices in corresponding to the subsets $S_1, S_2, \dots, S_l$. Therefore, there is a vertex cover of size $1 \leq l \leq k$.

Vertex Cover to Set Cover



$U = \{1, 2, 3, 4, 5, 6, 7, 8\}$

$S_1 = \{1, 2, 3\}$

$S_2 = \{1, 4\}$

$S_3 = \{3, 6\}$

$S_4 = \{4, 5, 6, 7\}$

$S_5 = \{2, 5\}$

$S_6 = \{7, 8\}$

$S_7 = \{8\}$

$\exists$ a **vertex cover** of size $k \Rightarrow \exists$ a **set cover** of size k

- If $\{u_1, u_2, \dots, u_k\}$ is a vertex cover in G , then by construction, $S_1 \cup S_2 \cup \dots \cup S_k = U$ (since U is the set of all edges).

Reduction

Warning

You should reduce the known NP-complete problem to the problem you are trying to prove NP-complete.

You will mistakenly do the opposite sometimes!

HAM cycle to TSP

HAM CYCLE $\rightarrow$ TSP

TSP $\rightarrow$ HAM CYCLE

We want to solve HAM CYCLE by using TSP

There is a TSP $\Rightarrow$ There is a HAM CYCLE

There is a HAM CYCLE $\Rightarrow$ There is a TSP

Do not forget to prove that TSP is in NP